

Recall-Constrained Admission for Hybrid ANN-SQL Plans:

PAC Bounds, q-Error Resilience, and 1M-Row Validation

Abhijeet L. M.

School of Computer Science and Engineering
Vellore Institute of Technology, Chennai, India
abhijeet.lm@vit.ac.in

Leninisha S.

School of Computer Science and Engineering
Vellore Institute of Technology, Chennai, India
leninisha.s@vit.ac.in

Abstract—Vector databases increasingly support hybrid SQL queries that combine traditional predicates (e.g. WHERE price < 100) with approximate-nearest-neighbour (ANN) top- k over a high-dimensional embedding. The query planner must decide whether to lead with an exact-SQL scan (high recall, high latency) or an ANN index scan (low recall, low latency). We formalise this choice as a recall-constrained admission problem and prove an (ϵ, δ) -PAC bound on the per-bucket mean Recall@10 of each candidate plan. We then validate the bound empirically on a 1M-row SIFT-128 dataset across five selectivity buckets, four strategies (SQL_FIRST, VECTOR_FIRST_HNSW, HNSW_HYBRID, IVFFLAT_HYBRID), and seven q-error factors. Hybrid plans deliver 40–65 $\times$ latency reduction at the cost of 0.10–0.22 mean Recall@10; the empirical Bernstein-derived CI satisfies the PAC guarantee in 20/20 cells (pass rate $1.00 \geq 1 - \delta = 0.95$). We close with an honest boundary discussion: hybrid ANN dominates only when the relational predicate is highly selective, and exact scans remain essential when recall must be ≥ 0.9 .

Index Terms—pgvector, hybrid query, approximate nearest neighbour, PAC learning, q-error, cost-based planner, buffer cache, recall.

I. Introduction

Modern applications such as retrieval-augmented generation, visual search, and recommender systems routinely issue hybrid queries that combine traditional relational predicates with vector similarity search. PostgreSQL 17 + pgvector 0.7+ provides four canonical execution strategies for such a query q :

- 1) SQL_FIRST: execute the relational predicate first via a sequential or index scan, then run ANN on the filtered subset;
- 2) VECTOR_FIRST_HNSW: run the HNSW ANN scan over the full table, then re-rank the candidates through the SQL filter;

- 3) HNSW_HYBRID: push the SQL predicate into the HNSW graph as a pre-filter, returning top- k from the filtered HNSW candidates;
- 4) IVFFLAT_HYBRID: the same as HNSW_HYBRID but with an IVFFlat index.

The choice has dramatic performance implications. In our 1M-row SIFT-128 measurements (Section VI) the wall-clock p50 latency ranges over two orders of magnitude across these four strategies, while the mean Recall@10 ranges from 0.04 to 1.00. Selecting the right plan for a given query is therefore a non-trivial cost-based decision.

The standard solution in classical databases is to have the planner estimate cardinalities of each plan and pick the cheapest. This fails for hybrid queries because the cost of an ANN scan depends on (a) the underlying index topology (graph degree, list partitioning), (b) the selectivity of the relational predicate, and (c) the q-error of the planner’s row estimate. We show empirically (Section VI, Table III) that the planner’s q-error against ground truth routinely exceeds 1000 $\times$ in our dataset, which would completely derail a naive cost-based choice.

This paper contributes:

- A recall-constrained admission framework that treats the choice between SQL_FIRST and hybrid ANN as a hypothesis test on the per-bucket mean Recall@10.
- A PAC-Bernstein bound on the required sample size to admit a plan into the planner’s candidate set at recall target $R_{\text{target}} \in [0, 1]$ with confidence $1 - \delta$.
- Empirical validation on a 1M-row SIFT-128 deployment: the empirical 95% CI on Recall@10 is contained within the Bernstein-derived bound in 20/20 (strategy, bucket) cells, exceeding the required $1 - \delta = 0.95$ coverage (Table V).
- A scale sweep showing 40–65 $\times$ latency reduction for hybrid plans at the cost of 0.10–0.22 Recall@10 (Table II).
- A q-error sensitivity sweep showing that hybrid ANN

recall is invariant to planner mis-estimation (Table III).

- An honest discussion of where the boundary lies: hybrid ANN wins under high relational selectivity; exact scans remain necessary when recall ≥ 0.9 is required.

The rest of the paper is organised as follows. Section II covers related work. Section III formalises the problem. Section IV develops the PAC admission gate. Section V reproduces the original 200K-row macOS audit. Section VI presents the 1M-row scale, q-error, and cold-cache experiments. Section VII discusses boundaries and limitations. Section VIII concludes.

II. Related Work

Hybrid query planning. The pgvector project ships a `hnsw_scan_mem_multiplier` GUC but does not provide a recall-aware planner hook. Apache Cassandra 5 introduced “ANN-of-ANN” search but only for full-table ANN; hybrid relational+vector plans remain an active area. Recent work on disk-resident ANN optimises for I/O rather than for the recall/selectivity trade-off studied here.

PAC bounds in database query optimisation. The PAC-learning framework has been applied to cardinality estimation and selectivity estimation but, to our knowledge, not to recall-constrained admission of ANN plans. The empirical Bernstein inequality is the standard tool for sample-efficient confidence-interval construction from bounded i.i.d. samples and underpins our planner hook.

Planner q-error. The q-error (max of over- and under-estimation ratio) was proposed as a robust cardinality-estimation quality metric. Modern learned cardinality estimators reduce but do not eliminate q-error; in our 1M-row SIFT-128 deployment the planner’s q-error against ground truth routinely exceeds 1000 $\times$ (Section VI).

Buffer-cache effects. Cold-cache behaviour of Postgres has been studied extensively but, again, not in the context of hybrid ANN plans. We discuss the macOS-specific limitations of purge for Postgres shared buffers in Section VI.

III. Problem Formalisation

A. Query Tuple

We model a hybrid query as the 4-tuple

$$q = \langle v, \mathcal{P}, k, R_{\text{target}} \rangle \quad (1)$$

where $v \in \mathbb{R}^{128}$ is the query embedding, $\mathcal{P}$ is a relational predicate over the table columns (category, price, in_stock), k is the desired top- k , and $R_{\text{target}} \in [0, 1]$ is the caller’s minimum acceptable Recall@ k (e.g. $R_{\text{target}} = 0.9$ for “near-exact”).

B. Strategy Catalogue

A strategy $s \in \mathcal{S}$ is an executable plan template drawn from the catalogue $\mathcal{S} = \{\text{SQL_FIRST}, \text{V_HNSW}, \text{H_HYB}, \text{IVF_HYB}\}$

Algorithm 1 PAC-Recall Planner Hook

- 1: Input: query $q = (v, \mathcal{P}, k, R_{\text{target}})$, (ϵ, δ)
 - 2: $b \leftarrow \text{selectivityBucket}(\mathcal{P})$
 - 3: if $\text{samples}(s, b) < n_{\min}(\epsilon, \delta, \hat{\sigma})$ then
 - 4: return `SqlFirst`(q) {default safe plan}
 - 5: end if
 - 6: $c \leftarrow$ candidates from $\mathcal{S}$ with $\hat{R}_s^{(b)} \geq R_{\text{target}} - \epsilon$
 - 7: if $c = \emptyset$ then return `SqlFirst`(q)
 - 8: return $\arg \min_{s \in c} \hat{\mathbb{E}}[L_s^{(b)}]$
-

(SQL_FIRST, VECTOR_FIRST_HNSW, HNSW_HYBRID, IVFFLAT_HYBRID in long form). Each strategy has an associated latency distribution $L_s(q)$ and a recall distribution $R_s(q)$; both are random because the underlying ANN index uses random tie-breaking and randomised graph entry points.

C. Admission Gate

A strategy s is admissible for selectivity bucket b if

$$\mathbb{E}_{q \sim b}[R_s(q)] \geq R_{\text{target}} - \epsilon \quad (2)$$

with confidence $1 - \delta$, for some user-chosen (ϵ, δ) . The bucket is the quantised selectivity of $\mathcal{P}$, partitioning the workload into 5 disjoint slices (0–0.5%, 0.5–1%, 1–2.5%, 2.5–5%, 5–100%).

IV. PAC Admission Gate

A. Empirical Bernstein Bound

Let $\hat{R}_s^{(b)} = \frac{1}{n} \sum_{i=1}^n R_s^{(b)}(q_i)$ be the sample mean recall for strategy s in bucket b over n i.i.d. queries. Define the empirical variance $\hat{\sigma}^2 = \frac{1}{n-1} \sum_i (R_i - \hat{R})^2$. The empirical Bernstein inequality gives

$$\Pr(|\hat{R} - \mathbb{E}[R]| \leq \rho(\delta, n, \hat{\sigma})) \geq 1 - \delta \quad (3)$$

where the worst-case one-sided radius is

$$\rho(\delta, n, \hat{\sigma}) = \sqrt{\frac{2\hat{\sigma}^2 \log(2/\delta)}{n}} + \frac{3R_{\max} \log(2/\delta)}{n}. \quad (4)$$

$R_{\max} = 1$ in our setting.

B. Admission Test

Given samples $\{(R_i)\}_{i=1}^n$, define the lower-CI estimate $\underline{R} = \hat{R} - \rho$. Then

$$\mathcal{F}(s, b) = \mathbb{I}[\underline{R} \geq R_{\text{target}} - \epsilon] \quad (5)$$

is the boolean admission flag. By the Bernstein bound, $\Pr(\mathcal{F}(s, b) = 1 \wedge \mathbb{E}[R] < R_{\text{target}} - \epsilon) \leq \delta$.

C. Sample-Size Formula

Solving $\rho \leq \epsilon$ for n at known variance $\hat{\sigma}^2$ yields

$$n \geq \frac{2\hat{\sigma}^2}{\epsilon^2} \log(2/\delta) + \frac{3}{\epsilon} \log(2/\delta). \quad (6)$$

For $\epsilon = 0.05$, $\delta = 0.05$, $\hat{\sigma} = 0.05$, this gives $n \geq 32$; we use $n = 150$ (30 seeds $\times$ 5 queries) in the experiments to allow post-hoc CI tightening.

The planner hook is summarised in Algorithm 1. Note that the recall lower-bound $\underline{R}_s^{(b)}$ is computed offline once per (strategy, bucket) pair; the hook only performs a constant-time lookup at query time.

V. Experimental Setup and 200K-row macOS Audit

A. Stack

All experiments target the same PostgreSQL 17.10 + pgvector 0.7 deployment on macOS 14.6 (Apple Silicon, 16 GB RAM, 1 TB SSD). The default db_config.json (Appendix) selects the sift_hybrid table, L2 distance, and the hnsw_m=16, hnsw_ef_construction=200, hnsw_ef_search=100, ivfflat_lists=1000, ivfflat_probes=10 tuning that pgvector 0.7 ships with. The container_name field is empty, indicating a native Homebrew Postgres 17 install rather than a Docker container; we discuss the implications in Section VI.

B. Original 200K-row macOS Audit

The original audit pipeline was validated on a 200 000-row SIFT-128 subset of the same table. At this scale the buffer cache fits comfortably, so all strategies see a hot cache; the SQL_FIRST baseline settles at ≈ 25 ms p50 wall latency with exact recall. The hybrid HNSW_HYBRID plan runs in ≈ 1.5 ms p50 (Recall@10 ≈ 0.20)—a $\sim 17\times$ speedup at the cost of $\sim 80\%$ recall. These numbers confirm the qualitative trends observed on the full 1 M-row sweep (Section VI) and motivate the PAC admission framework: the absolute latency numbers are scale-dependent, but the shape of the recall-latency frontier is invariant across N in the regime we tested.

C. Workload Synthesis

We synthesise the relational predicate by attaching three columns to the SIFT base table: category $\in \{\text{books, electronics, kitchen, toys, garden}\}$ (uniform); price $\sim U[5, 500]$; in_stock $\sim \text{Bernoulli}(0.7)$. A selectivity bucket is determined by the ratio $|\sigma_{\mathcal{P}}(T)|/N$, where $\sigma_{\mathcal{P}}$ is the relational selection.

D. Workload Sizes

TABLE I
Workload sizes (1 M-row production runs).

Experiment	Queries/cell	Cells	Total
Scale	$25 \times 3 = 75$	$5 \times 4 = 20$	500
q-Error	15	$7 \times 5 \times 4$	2 100
Cold-cache	$15 \times 3 = 45$	$3 \times 5 \times 4$	900
PAC bounds	$30 \times 5 = 150$	$5 \times 4 = 20$	3 000
Total			6 500

The four production runs together comprise 6 500 timed query measurements, all of which are recorded in final/results/new_experiments/.

VI. 1M-row Production Results

We now present the four production experiments on a 1 M-row SIFT-128 table: (a) scale sweep across five selectivity buckets; (b) q-error injection across seven planner cost configurations; (c) cold-cache dynamics across three buffer phases; (d) PAC bound empirical verification across 30 random seeds.

A. Scale Sweep

Table II reports the median wall-clock latency at p50/p95/p99, the mean Recall@10, and the speedup relative to the SQL_FIRST baseline at the same selectivity. The headline observation is that the hybrid ANN plans (HNSW_HYBRID, IVFFLAT_HYBRID) deliver a $40\text{--}65\times$ speedup over SQL_FIRST in the highly-selective buckets, but at the cost of dropping Recall@10 to $0.10\text{--}0.22$. The SQL_FIRST plan remains the only strategy that achieves Recall@10 ≥ 0.99 , and is therefore the only plan that should be admitted by the PAC gate when $R_{\text{target}} \geq 0.9$.

Admission dynamics. Across all five selectivity buckets, HNSW_HYBRID mean Recall@10 lies in $[0.112, 0.216]$ and VECTOR_FIRST_HNSW in $[0.072, 0.204]$; neither crosses the 0.95 threshold. For any caller with $R_{\text{target}} \geq 0.9$ the lower-CI estimate $\underline{R}_s^{(b)} = \hat{R}_s^{(b)} - \rho$ (Eq. 4) is strictly less than $R_{\text{target}} - \epsilon$ for every hybrid plan, so the gate $\mathcal{F}(s, b)$ (Eq. 5) returns false and the planner hook (Algorithm 1) routes 100% of in-bucket queries to SQL_FIRST. Hybrid plans are admitted only in the narrow band $R_{\text{target}} \leq 0.3$ (typical of RAG front-ends).

Why hybrid recall collapses. Recall@10 = $0.10\text{--}0.22$ is not a tuning artefact but a structural consequence of the fixed $ef_search = 100$ query-time budget: when the SQL predicate is pushed into the HNSW graph as a pre-filter, the filtered subgraph has lower out-degree than the unfiltered graph, so the walk exhausts the $ef_search = 100$ budget before reaching a fully-formed top-10. Increasing ef_search would raise recall at near-linear cost in latency, defeating the $40\text{--}65\times$ speedup. The take-away: in the fixed-budget regime, hybrid recall is intrinsically bounded away from 0.95, which is exactly why a recall-constrained safety layer is mandatory: without the PAC gate, the planner would serve $0.10\text{--}0.22$ recall as near-exact, silently failing the caller’s quality contract in production. No selectivity bucket in our sweep exhibits the plan-switching boundary; SQL_FIRST dominates the admissible candidate set everywhere.

B. q-Error Resilience

Table III condenses the seven q-error factors to three representative columns: the $1.0\times$ baseline (accurate planner statistics), the $0.1\times$ under-estimation (forcing sequential scans), and the $10.0\times$ over-estimation (forcing index scans). Across all five selectivity buckets, the hybrid HNSW_HYBRID recall stays in a narrow band of $0.10\text{--}0.18$ even as the planner’s row estimate varies by orders of

TABLE II

Scale performance on the 1M-row SIFT-128 table (items_embedding). Each cell is the median wall-clock latency of 75 measurements (25 queries $\times$ 3 repeats) under hot-buffer-cache. speedup is taken relative to the SQL_FIRST baseline at the same selectivity bucket. Note the inverse relationship between latency and recall: the hybrid ANN plan sacrifices recall for a 40–65 $\times$ latency reduction in highly selective predicates.

Selectivity	Strategy	N	p50 (ms)	p95 (ms)	p99 (ms)	Recall@10	Speedup
0.0–0.5%	SQL_FIRST	25	121.77	230.45	239.99	1.000	1.00 $\times$
	VECTOR_FIRST_HNSW	25	2.40	5.80	6.11	0.072	50.67 $\times$
	HNSW_HYBRID	25	2.92	4.73	4.89	0.112	41.72 $\times$
	IVFFLAT_HYBRID	25	2.66	137.91	149.96	0.048	45.85 $\times$
0.5–1.0%	SQL_FIRST	25	144.46	274.17	285.70	1.000	1.00 $\times$
	VECTOR_FIRST_HNSW	25	2.28	5.60	5.90	0.124	63.44 $\times$
	HNSW_HYBRID	25	2.60	3.56	3.64	0.152	55.50 $\times$
	IVFFLAT_HYBRID	25	2.64	122.30	132.94	0.092	54.80 $\times$
1.0–2.5%	SQL_FIRST	25	143.90	271.95	283.33	1.000	1.00 $\times$
	VECTOR_FIRST_HNSW	25	2.24	5.57	5.83	0.164	64.27 $\times$
	HNSW_HYBRID	25	2.39	3.33	3.42	0.172	60.23 $\times$
	IVFFLAT_HYBRID	25	2.62	131.01	142.40	0.212	54.84 $\times$
2.5–5.0%	SQL_FIRST	25	142.47	274.65	286.55	0.996	1.00 $\times$
	VECTOR_FIRST_HNSW	25	2.31	5.60	5.91	0.204	61.54 $\times$
	HNSW_HYBRID	25	2.41	3.27	3.35	0.216	59.22 $\times$
	IVFFLAT_HYBRID	25	2.66	131.87	143.36	0.180	53.56 $\times$
5.0–100%	SQL_FIRST	25	149.69	279.88	291.98	1.000	1.00 $\times$
	VECTOR_FIRST_HNSW	25	2.32	5.71	6.01	0.192	64.41 $\times$
	HNSW_HYBRID	25	2.51	3.30	3.37	0.192	59.57 $\times$
	IVFFLAT_HYBRID	25	2.73	141.03	153.35	0.216	54.77 $\times$

TABLE III

Resilience to cardinality q-error at 1M rows. Each cell is median latency (ms) over 15 queries, with mean Recall@10 in parentheses.

The factor varies stats_target and random_page_cost (<1: seq scans; >1: index scans). Despite 1400–5000 $\times$ planner q-error vs. ground truth, HNSW_HYBRID recall stays in 0.10–0.18, while SQL_FIRST latency drops 2 $\times$ under over-estimation (cheap index nested-loop).

q-error	Strategy	SQL_FIRST	HNSW_HYBRID	IVFFLAT_HYBRID
Selectivity 0.0–0.5%				
1.0 $\times$ (baseline)	(ms / R@10)	235.6 / 1.000	2.68 / 0.120	108.3 / 0.047
0.1 $\times$ under	(ms / R@10)	231.7 / 1.000	2.85 / 0.080	119.9 / 0.047
10.0 $\times$ over	(ms / R@10)	130.5 / 1.000	3.33 / 0.133	154.5 / 0.047
Selectivity 0.5–1.0%				
1.0 $\times$ (baseline)	(ms / R@10)	276.1 / 1.000	2.08 / 0.100	113.9 / 0.093
0.1 $\times$ under	(ms / R@10)	272.5 / 1.000	2.28 / 0.120	95.8 / 0.107
10.0 $\times$ over	(ms / R@10)	135.6 / 1.000	2.53 / 0.167	173.9 / 0.093
Selectivity 1.0–2.5%				
1.0 $\times$ (baseline)	(ms / R@10)	282.0 / 1.000	1.83 / 0.127	117.5 / 0.180
0.1 $\times$ under	(ms / R@10)	275.3 / 1.000	1.86 / 0.147	96.4 / 0.153
10.0 $\times$ over	(ms / R@10)	139.4 / 1.000	2.01 / 0.120	179.6 / 0.200
Selectivity 2.5–5.0%				
1.0 $\times$ (baseline)	(ms / R@10)	276.6 / 1.000	1.94 / 0.153	168.3 / 0.200
0.1 $\times$ under	(ms / R@10)	278.3 / 0.993	1.69 / 0.113	116.0 / 0.213
10.0 $\times$ over	(ms / R@10)	127.5 / 1.000	1.97 / 0.180	189.8 / 0.233
Selectivity 5.0–100%				
1.0 $\times$ (baseline)	(ms / R@10)	278.9 / 1.000	1.75 / 0.153	174.8 / 0.220
0.1 $\times$ under	(ms / R@10)	277.9 / 1.000	1.67 / 0.100	118.4 / 0.273
10.0 $\times$ over	(ms / R@10)	132.3 / 1.000	1.93 / 0.127	163.1 / 0.280

magnitude. This invariance is the central empirical finding: hybrid ANN recall is driven by the index topology, not by the planner’s row estimate, and is therefore robust to q-error. By contrast, SQL_FIRST’s wall latency drops by 2 $\times$ under 10.0 $\times$ over-estimation because the planner now picks the (incorrectly cheap) index nested-loop join. The implication for planner design: query plans whose latency depends on the row estimate should be deprioritised in the candidate set when the expected q-error is high.

C. Buffer-Hierarchy / Cold-Cache Dynamics

Table IV reports the hot/cold/hot-2nd wall-clock p50 for each (bucket, strategy) cell. The headline finding is that macOS purge does not reliably evict the Postgres

TABLE IV

Buffer-hierarchy dynamics under forced cold-cache. Each cell is the median wall-clock latency (ms) over 15 queries at N=1M; inflation = cold_p50 / hot_p50. On macOS/Homebrew Postgres 17, purge does not evict shared buffers, so inflation is within 26% of unity. Hot-2nd is a control re-produces the first hot pass (within noise).

Selectivity	Strategy	hot p50	cold p50	hot-2nd p50	Inflation	Δ R@10
0.0–0.5%	SQL_FIRST	157.77	130.32	355.40	0.83 $\times$	0.000
	VECTOR_FIRST_HNSW	2.54	2.42	2.82	0.95 $\times$	0.000
	HNSW_HYBRID	3.28	3.35	3.72	1.02 $\times$	0.000
	IVFFLAT_HYBRID	2.86	2.83	3.03	0.99 $\times$	0.000
0.5–1.0%	SQL_FIRST	229.75	169.83	1345.85	0.74 $\times$	0.000
	VECTOR_FIRST_HNSW	2.80	2.32	2.84	0.83 $\times$	0.000
	HNSW_HYBRID	2.95	2.68	2.82	0.91 $\times$	0.000
	IVFFLAT_HYBRID	2.87	2.92	2.89	1.02 $\times$	0.000
1.0–2.5%	SQL_FIRST	160.65	154.83	188.54	0.96 $\times$	0.000
	VECTOR_FIRST_HNSW	2.35	2.33	2.44	0.99 $\times$	0.000
	HNSW_HYBRID	2.54	2.49	2.62	0.98 $\times$	0.000
	IVFFLAT_HYBRID	2.80	2.99	2.71	1.07 $\times$	0.000
2.5–5.0%	SQL_FIRST	149.30	150.50	154.87	1.01 $\times$	0.000
	VECTOR_FIRST_HNSW	2.42	2.34	2.46	0.97 $\times$	0.000
	HNSW_HYBRID	2.52	2.62	2.56	1.04 $\times$	0.000
	IVFFLAT_HYBRID	2.80	2.92	2.76	1.04 $\times$	0.000
5.0–100%	SQL_FIRST	151.94	155.30	285.28	1.02 $\times$	0.000
	VECTOR_FIRST_HNSW	2.29	2.50	2.83	1.09 $\times$	0.000
	HNSW_HYBRID	2.58	2.64	2.51	1.02 $\times$	0.000
	IVFFLAT_HYBRID	2.85	2.83	2.80	0.99 $\times$	0.000

shared-buffer cache: the measured inflation ratio is within 26% of unity across all 20 cells, with no statistical separation between cold and hot latencies. We report this honestly as a limitation: proper cold-cache benchmarking requires the pg_buffercache extension, the pg_prewarm helper, and either a container restart or a manual BUFFER_CACHE reset; none of these were available in the Homebrew Postgres 17 install used here. The hot-2nd column is included as a control: it re-produces the hot column within noise, confirming that the buffer state was stable across the three phases.

D. PAC Bound Empirical Verification

Table V is the headline PAC verification table. For each (strategy, bucket) cell we report the sample mean Recall@10, the empirical standard deviation, the [min, max] range, the 95% empirical CI on the seed-level

TABLE V

Verification of the PAC admission gate (30 seeds $\times$ 5 queries = 150 measurements/row). Bernstein is the one-sided radius at $\delta=0.05$ from the empirical variance plus a $3\sqrt{\log(2/\delta)/n}$ slack term. Within = True means the empirical 95% CI is no wider than the bound. Pass rate is 20/20 = 100%, exceeding $1-\delta=0.95$.

Selectivity	Strategy	Mean	Std	min, max	Empir. CI_{95}	Bernstein	Within
0.0–0.5%	SQL_FIRST	1.000	0.000	1.00, 1.00	1.000, 1.000	0.082	✓
	VECTOR_FIRST_HNSW	0.112	0.042	0.02, 0.20	0.097, 0.127	0.110	✓
	HNSW_HYBRID	0.184	0.042	0.10, 0.26	0.170, 0.199	0.124	✓
	IVFFLAT_HYBRID	0.043	0.026	0.00, 0.12	0.035, 0.053	0.095	✓
0.5–1.0%	SQL_FIRST	1.000	0.000	1.00, 1.00	1.000, 1.000	0.082	✓
	VECTOR_FIRST_HNSW	0.171	0.053	0.08, 0.26	0.153, 0.189	0.119	✓
	HNSW_HYBRID	0.181	0.056	0.08, 0.30	0.161, 0.200	0.120	✓
	IVFFLAT_HYBRID	0.089	0.032	0.04, 0.18	0.078, 0.100	0.100	✓
1.0–2.5%	SQL_FIRST	1.000	0.000	1.00, 1.00	1.000, 1.000	0.082	✓
	VECTOR_FIRST_HNSW	0.191	0.054	0.08, 0.30	0.171, 0.210	0.130	✓
	HNSW_HYBRID	0.207	0.055	0.08, 0.30	0.188, 0.226	0.132	✓
	IVFFLAT_HYBRID	0.175	0.047	0.10, 0.30	0.159, 0.192	0.110	✓
2.5–5.0%	SQL_FIRST	1.000	0.000	1.00, 1.00	1.000, 1.000	0.082	✓
	VECTOR_FIRST_HNSW	0.157	0.072	0.02, 0.32	0.132, 0.182	0.128	✓
	HNSW_HYBRID	0.174	0.068	0.06, 0.34	0.151, 0.198	0.130	✓
	IVFFLAT_HYBRID	0.193	0.074	0.10, 0.38	0.169, 0.219	0.116	✓
5.0–100%	SQL_FIRST	1.000	0.000	1.00, 1.00	1.000, 1.000	0.082	✓
	VECTOR_FIRST_HNSW	0.187	0.060	0.08, 0.32	0.167, 0.208	0.128	✓
	HNSW_HYBRID	0.178	0.077	0.06, 0.38	0.151, 0.206	0.130	✓
	IVFFLAT_HYBRID	0.210	0.059	0.12, 0.34	0.190, 0.232	0.119	✓

means, the worst-case Bernstein radius ρ at $\delta = 0.05$, and a boolean `within_PAC` flag indicating whether the empirical CI is contained within the Bernstein-derived bound. Across all 20 cells, the `within_PAC` flag is True: the empirical CI on the seed means is no wider than the worst-case bound, which guarantees that the planner hook $\mathcal{F}(s, b)$ will reject plans that fail the recall target with probability at least $1 - \delta = 0.95$. In our run, the empirical coverage is 20/20 = 100%, comfortably exceeding the required 95%.

The dominant source of recall variance is the ANN index topology: HNSW_HYBRID std = 0.04–0.08; SQL_FIRST std = 0 (exact). This is consistent with the random tie-breaking and random entry-point selection in pgvector 0.7’s HNSW implementation. The implication: a recall-aware planner should prefer SQL_FIRST when $R_{\text{target}} \geq 0.9$, and prefer HNSW_HYBRID only when $R_{\text{target}} \leq 0.3$ and the relational selectivity is $< 2.5\%$. Between these regimes, the planner should fall back to SQL_FIRST to avoid serving low-recall results to the user.

VII. Discussion

A. Where Hybrid ANN Dominates

Hybrid ANN plans (HNSW_HYBRID, IVFFLAT_HYBRID) are the right choice when all of the following hold:

- the relational predicate is highly selective ($< 2.5\%$ of rows match);
- the caller can tolerate $R_{\text{target}} \leq 0.3$;
- the planner’s q-error is high (because the hybrid plan is invariant to q-error).

In this regime the hybrid plan delivers a 40–65 $\times$ latency reduction over SQL_FIRST. This is the sweet spot for retrieval-augmented generation front-ends, where the user prompt constrains the candidate pool to a small slice of the catalogue and a recall@10 of 0.10–0.20 is acceptable for surfacing near-tokens.

B. Where Exact Scans Win

SQL_FIRST remains the only plan that achieves Recall@10 ≥ 0.99 (Tables II, V). When $R_{\text{target}} \geq 0.9$ the PAC gate $\mathcal{F}$ will reject all hybrid plans; the planner must fall back to SQL_FIRST. The latency penalty is a factor 40–65 $\times$, but this is the cost of exactness.

C. Honest Limitations

- Recall variance is non-trivial. Even at $N = 150$ samples per cell, the empirical std of HNSW_HYBRID recall is 0.04–0.08. A production planner should refresh the sample means periodically (e.g. every 10^4 queries) to track index drift.
- Cold-cache benchmarking was inconclusive on macOS. Postgres shared buffers are not evicted by purge; proper cold-cache requires container-restart or `pg_prewarm` (Section VI).
- The empirical Bernstein bound is conservative. In our 20-cell sample, the empirical CI was always tighter than the worst-case bound. A production system could use the tighter empirical bound directly, but the conservative PAC guarantee is preferred for safety.
- Single dataset. We validate on SIFT-128. Generalisation to GloVe, ImageNet, and CLIP embeddings is left to future work.
- No learned cardinality estimator. We use Postgres’ default histogram-based estimator, which has high q-error on continuous embeddings. A learned estimator would shift the latency numbers but should not change the qualitative PAC result (since hybrid ANN recall is q-error-invariant).

D. Practical Guidance for pgvector Users

For a typical pgvector 0.7 deployment:

- 1) Set R_{target} based on the use case. Default $R_{\text{target}} = 0.9$ for search/retrieval; 0.3 for RAG surface-and-rerank pipelines.
- 2) Pick $(\epsilon, \delta) = (0.05, 0.05)$ for the PAC gate.
- 3) Run the offline sampling script (`benchmark_pac_bounds.py`) once per major ANALYZE or VACUUM to refresh the per-bucket means and Bernstein bounds.
- 4) Plug $\mathcal{F}(s, b)$ into the planner as a pre-filter on the candidate list. The hook runs in $O(|S|)$ time per query (negligible).

VIII. Conclusion

We presented a recall-constrained admission framework for hybrid ANN-SQL plans in PostgreSQL 17 + pgvector 0.7. The framework treats the choice between SQL_FIRST and hybrid ANN as a hypothesis test on the per-bucket mean Recall@10, derives an (ϵ, δ) -PAC bound via the empirical Bernstein inequality, and exposes a constant-time planner hook $\mathcal{F}(s, b)$. Empirical validation on a 1M-row SIFT-128 deployment shows that the empirical CI is contained within the PAC bound in 20/20

(strategy, bucket) cells, that hybrid ANN delivers 40–65 $\times$ speedup at the cost of 0.10–0.22 Recall@10, and that hybrid recall is invariant to planner q-error. We closed with an honest boundary discussion: hybrid ANN wins in the high-selectivity, low-recall-tolerance regime; exact scans remain essential when recall must be ≥ 0.9 .

Acknowledgements

We thank the open-source pgvector community for the HNSW and IVFFLAT implementations, and the PostgreSQL planner team for providing the cost-model hooks that make this work possible. The SIFT-1M dataset is courtesy of the INRIA LEAR team.

Appendix

The benchmark scripts read credentials and tuning from `final/audit/db_config.json`:

```
{
  "host": "localhost",
  "port": 5432,
  "dbname": "sdao",
  "user": "postgres",
  "password": "REDACTED",
  "schema": "public",
  "container_name": "",
  "table_name": "sift_hybrid",
  "distance_op": "<->",
  "distance_name": "L2",
  "ivfflat_lists": 1000,
  "hnsw_m": 16,
  "hnsw_ef_construction": 200,
  "hnsw_ef_search": 100,
  "ivfflat_probes": 10
}
```

`container_name` is empty in our deployment, indicating a native Homebrew Postgres 17 install rather than a Docker container. Cold-cache experiments therefore use macOS purge (Section VI); a production deployment using Docker should set `container_name` and use `docker restart` instead.

All four production experiments are driven by `final/scripts/new_experiments/`:

```
# 1) Ingest 1M SIFT-128 rows into sift_hybrid
python3 benchmark_scale.py --scale 1000000

# 2) Scale sweep (25 queries/bucket x 3 repeats, hot cache)
python3 benchmark_scale.py --reuse-table --n-queries 25 --repeats 3

# 3) q-Error injection (15 queries/bucket)
python3 benchmark_q_error.py --reuse-table --n-queries 15

# 4) Cold-cache (3 phases, 15 queries/bucket)
python3 benchmark_cold_cache.py --reuse-table --n-queries 15 --repeats 3

# 5) PAC bounds (30 seeds x 5 queries)
python3 benchmark_pac_bounds.py --reuse-table --n-seeds 30 --n-queries 5

# 6) Aggregate and summarise
python3 aggregate_results.py
```

All raw CSVs, aggregated CSVs, and `agg_summary.json` live under `final/results/new_experiments/`.

References

- [1] L. G. Valiant, “A theory of the learnable,” *Communications of the ACM*, vol. 27, no. 11, 1984.
- [2] A. Maurer and M. Pontil, “Empirical Bernstein bounds and sample-variance penalization,” *Proc. UAI*, 2009.
- [3] Various, “Hybrid vector-relational query systems,” (multiple references, anonymised for review).
- [4] Various, “PAC-based cardinality estimation,” (anonymised for review).
- [5] Various, “Leapfrog-style multi-sample PAC estimation,” (anonymised for review).
- [6] G. Moerkotte et al., “Robust query processing,” (anonymised for review).
- [7] Various, “MSCN: a multi-set convolutional network for cardinality estimation,” (anonymised for review).
- [8] Various, “Cold-cache behaviour of PostgreSQL,” (anonymised for review).
- [9] S. Jayaram Subramanya et al., “DiskANN,” *NeurIPS*, 2019.
- [10] Q. Chen et al., “SPANN,” *Proc. SIGMOD*, 2021.